

System Design Document — Socure Lead Pipeline

Companion to the architecture one-pager. This is the understanding doc: what each component does, the contracts between them, and the decisions behind them.

1 · Goals and non-goals

Goals: enrich, score, and route 30 leads with full provenance; deterministic, reproducible decisions; every output carries its reason; runnable with zero keys.

Non-goals (documented extensions, not built): intent-signal scoring (mock CSV has none), learning loop retuning weights from the rep ledger, CRM write-back, Slack delivery (production rail, spec'd in `IMPLEMENTATION.md`).

2 · System overview

Two entry points run the same library code:

- `npm run pipeline` → `scripts/run-pipeline.ts` — local, zero keys, no LLM. Covers ingest → suppress → enrich (public+symbolic) → score → route → outputs.
- Trigger.dev tasks (`tools/`) — the same stages as retryable cloud tasks, plus the two LLM stages (battle cards, brief narration), which need `ANTHROPIC_API_KEY`.

That split is deliberate: **the core never depends on a key, a vendor, or a model**. The LLM stages bolt on; they never sit in the core's path.

3 · Component design

Component	Responsibility	Key decision
<code>lib/csv.ts</code>	Parse/serialize CSV	Identity fields never mutated after ingest
<code>lib/suppression.ts</code>	Gate 0: build customer-domain set, test membership	43 customer accounts → 34 clean registrable domains; match on lead domain OR email domain; suppressed leads go to the expansion book, never dropped
<code>lib/domain.ts</code>	Registrable-domain normalization	One definition shared by suppression, grouping, routing — no drift between stages

<code>lib/enrich.ts</code>	Cascade: Bitscale → nRev → DNS MX/RDAP → symbolic → LLM residue; <code>assertEnrichmentContract</code>	Residue classifies, never scores; confidence code-clamped (≤ 0.6); agreement between independent methods raises trust, disagreement keeps the LLM's ceilinged confidence; unparseable LLM output → keep symbolic, never block
<code>lib/scoring.ts</code>	4-dimension rubric over <code>config/icp-rubric.json</code>	Pure function; vertical matches <code>domain +</code> <code>enrichment.industry</code> with first-bucket-wins priority, so domain evidence outranks LLM labels; DQ flags attach reasons, never subtract points
<code>lib/routing.ts</code>	<code>groupAccounts</code> (registrable domain, score = MAX, committee depth) + <code>routeAccount</code>	Account-first: the buyer is a committee. MAX not sum (sum rewards spam) not avg (an IT manager dilutes a CRO). Ownership beats territory; a stale owner falls through to fresh assignment
<code>lib/ownership.ts</code>	Ledger: <code>{domain → rep_id,</code> <code>assigned_at, run_id}</code> in <code>memory/account-owners.json</code>	New assignments persist; re-runs route identically — the ledger is what makes runs idempotent at the routing layer
<code>connections/*</code>	Bitscale, nRev, public DNS/RDAP, Anthropic+Langfuse clients	Vendors env-gated + probe-first; every client degrades to null, never throws into the pipeline
<code>tools/battle-</code> <code>card.ts</code>	LLM card per hot/warm account, per <code>prompts/battle-card.md</code>	Grounding corpus only; Langfuse groundedness eval; rep gate downstream
<code>tools/decision-</code> <code>brief.ts</code>	Compute deltas/trust/projections vs. <code>memory/runs/</code> ; LLM narrates	Code computes every number; narration optional and env-gated

4 • Data model (`lib/types.ts`)

```

Lead    { id, name, email, domain, title,           // identity – never mutated
          enrichment?, score?, routing?, suppressed?, battle_card_path? }
Enrichment { industry, sub_vertical, employee_range, est_revenue, tech_signals[],
             source: bitscale|nrev|public+symbolic|symbolic|llm, confidence }
Score    { total, breakdown{dim → {bucket, points}}, tier: hot|warm|cold, dq_reason? }
Routing  { rep_id, rep, priority, sla, owner_status: existing_owner|new_assignment, reason }
Account  { domain, leads[], account_score (MAX), tier, committee_depth,
           existing_customer, routing? }

```

The `Lead` accretes — each stage adds a block, none rewrites a previous one. The `Account` is the routing unit; leads inherit their account's routing. The same `Account` shape serves the Part 2 expansion book.

5 • Run sequence (from `scripts/run-pipeline.ts`)

1. Ingest 30 leads; build suppression set; split acquisition vs. expansion.
2. Enrich all acquisition leads concurrently; assert the data contract per lead; score each (pure function, safe in parallel).
3. Group into accounts; route each account against the ownership ledger; leads inherit; new assignments written back to the ledger.
4. Write `output/routed_leads.csv` + `.json` (accounts + expansion book).
5. (Trigger.dev only) battle cards for hot/warm; decision brief vs. prior snapshot.

6 • Invariants and error handling

Enforced in code, tested offline by `npm run check` (no network, no keys):

- 30 leads in, 30 accounts out, **zero unrouted** (unmatched vertical → Rep C, flagged).
- Every `breakdown` sums to its `total`; sentinel leads pin the rubric (VP Fraud & Risk @ financial ≥85 hot; manufacturing and .edu cold **with** `dq_reason`).
- Suppression: known-customer domain must suppress; no mock lead matches a real customer.
- Data contract: identity + full enrichment block per lead, or the run fails loudly.
- Degradation: any failed source lowers one lead's confidence; nothing blocks the run.
- Idempotency: ownership ledger + deterministic scoring → re-runs produce the same book, no duplicate rows/cards/events.

7 • Security & secrets

Keys live in `.env` locally (gitignored) and Trigger.dev env vars in prod — never in code or prompts. The zero-key path is the default; vendors and LLM activate only when their keys exist. No PII beyond the mock CSV enters any third party; enrichment requests send domain/email/title only.

8 • Observability

Trigger.dev: per-stage success/retries/duration, shareable run link. Langfuse: per-call prompt/tokens/cost/latency + groundedness eval on cards. Run-over-run: `memory/runs/` snapshots power the decision brief's deltas. Alert thresholds and cadence live in `OPERATIONS.md`.

9 · Alternatives considered

- **Lead-based routing** — rejected: Socure's buyer is a committee; account-first matches lead-to-account reality and prevents split ownership.
- **LLM-judged scoring** — rejected: scores feed tracked metrics; an LLM judge drifts week-over-week and makes deltas meaningless.
- **Always-on LLM enrichment** — rejected: the cascade puts free/cheap deterministic sources first; the LLM is last and bounded, so cost and failure modes stay small.
- **Monolithic task** — rejected: chained Trigger.dev tasks give per-stage retries and observability; that granularity is why Trigger.dev was picked at all.

Socure Growth Engineering Exercise · source of truth: [docs/SYSTEM-DESIGN.md](#)